

Christopher Contreras

cdc17507@csu.fullerton.edu

Pseudocode:

```
function minSwapsCouples(row)
    n = length(row) / 2
    index_map = {person: index for index, person in enumerate(row)}
    swaps = 0

    for i = 0 to length(row) step 2
        first_person = row[i]
        second_person = first_person XOR 1

        if row[i + 1] != second_person then
            partner_index = index_map[second_person]
            swap row[i + 1] with row[partner_index]
            update index_map[row[partner_index]] = partner_index
            update index_map[row[i + 1]] = i + 1
            swaps += 1

    return swaps
```

Induction, Limit, and Step Counts:

❖ Induction:

- The loop in the array works in step two because each pair will check whether or not the correct couple is sitting next to each other. If they do not sit next to each other, a swap will occur that updates the index_map. The number of steps that happen will ultimately depend on the number of swaps that occur for the couple to be next to each other.

❖ Step Counts:

- The loop runs from 0 to n (which is $\text{length}(\text{row}) / 2$), stepping by 2.
- The dictionary then looks at index_map, which updates the constant time to $O(1)O(1)O(1)$. The total number of operations inside the loop are as follows:

checking the condition, swapping the number, and checking the constant of each pair. Therefore, the loop runs $n/2$ times, with constant work per iteration.

Time Complexity:

- ❖ **Building the index map:** The map that is made is passed over a single input list that takes $O(n)$, meaning that n is the length of the row.
- ❖ **Loop through the list:** The loop will run for each pair, and within the loop, all the operations are $O(1)$ because of the constant time that the dictionary uses for lookups and swaps. There the time complexity is $O(n)$, where n is the length of input for the array row.